

DOCUMENTACIÓN TÉCNICA Y MANUAL DE IMPLEMENTACIÓN API REST

Plataforma de Gestión Logística Agrícola: "El Surco"

Fase del Proyecto: Desarrollo de Backend, Base de Datos y API RESTful **Versión:** 2.0 (Extendida)

1. EQUIPO DE DESARROLLO (CONSTANCIA DE PARTICIPACIÓN)

Se hace constar formalmente que el diseño, desarrollo, pruebas (testing) y depuración de la presente fase del proyecto fue realizado de manera equitativa y colaborativa por todos los miembros del equipo. Cada integrante cumplió al 100% con los requerimientos asignados durante los Sprints de desarrollo:

- **Pablo David Requena Mollinedo:** Scrum Master, Arquitectura de Backend y Resolución de Conflictos
 - **José Lionzo Xic Chay:** Desarrollo de Software, Modelado de Datos e Implementación de Servicios.
 - **Andrés Angel Emmanuel Gonzalez García:** Desarrollo de Software, Testing de Endpoints y Mapeo Relacional.
 - **Randolf Alfredo Bran Reyes:** Desarrollo de Software, Creación de Controladores y Pruebas en Postman.
-

2. RESUMEN EJECUTIVO DEL SISTEMA

"El Surco" es una solución tecnológica desarrollada para optimizar la cadena de suministro agrícola en Guatemala. El sistema provee una infraestructura digital (Backend) que permite el comercio directo entre productores (Agricultores) y consumidores (Compradores), eliminando la dependencia de intermediarios, asegurando el comercio justo y manteniendo un registro íntegro de inventarios, pedidos y facturación.

3. ARQUITECTURA DEL SISTEMA Y TECNOLOGÍAS

Para garantizar la escalabilidad, seguridad y eficiencia del sistema, se implementó una arquitectura basada en microservicios monolíticos utilizando las siguientes tecnologías:

- **Lenguaje de Programación:** Java (JDK 17+).
 - **Framework Principal:** Spring Boot 3.x.
 - **Gestor de Base de Datos:** MySQL Server 8.0 / MySQL Workbench.
 - **ORM (Object-Relational Mapping):** Hibernate y Spring Data JPA.
 - **Patrón de Diseño:** MVC (Model-View-Controller) adaptado a API REST (Capas: `Entity`, `Repository`, `Service`, `Controller`).
 - **Herramienta de Testing y Consumo de API:** Postman.
 - **Puerto de Despliegue Local:** `localhost:8081`
-

4. DISEÑO DE LA BASE DE DATOS (MODELO RELACIONAL)

Se diseñó y normalizó una base de datos relacional llamada `elsurcodb_in5bv`, garantizando la integridad referencial mediante el uso estricto de Llaves Primarias (PK) y Llaves Foráneas (FK). Las tablas implementadas son:

1. **Login:** Almacena credenciales de acceso (`email`, `password`) y restringe el acceso mediante la asignación de un `rolUsuario` (AGRICULTOR o COMPRADOR).
 2. **Categoria:** Catálogo clasificador para el inventario (ej. Granos Básicos, Frutas).
 3. **Agricultor:** Entidad que registra datos personales, DPI, contacto y ubicación GPS del productor. Se relaciona 1:1 con la tabla `Login`.
 4. **Comprador:** Entidad que registra los datos del cliente y su `direccionEntrega`. Se relaciona 1:1 con la tabla `Login`.
 5. **Producto:** Núcleo del inventario. Registra nombre, descripción, precio, stock y fecha de cosecha. Posee relaciones FK con `Agricultor` (quién lo vende) y `Categoria` (qué tipo de producto es).
 6. **Pedido:** Entidad transaccional que registra la compra. Vincula un `Comprador` con un `Producto`, calculando el total y la cantidad bajo un estado de flujo (ej. PENDIENTE).
 7. **Factura:** Emite el comprobante fiscal de la transacción, vinculando el `NIT` y la fecha de emisión directamente al ID del `Pedido`.
 8. **Valoracion:** Sistema de reputación donde el `Comprador` asigna una puntuación (1 a 5) y un comentario a un `Producto` adquirido, incluyendo la fecha exacta de la reseña (`fechaValoracion`).
-

5. IMPLEMENTACIÓN DE LA API RESTful (ENDPOINTS)

El sistema expone una API REST completa para el consumo de datos mediante formato **JSON**. A continuación, se detalla la documentación técnica de los Endpoints principales desarrollados para el flujo operativo (CRUD):

5.1 Módulo de Seguridad y Usuarios

- **Crear Credenciales:**
 - `POST /api/login/post`
 - **Body:** emailLogin, passwordLogin, rolUsuario
- **Crear Perfil de Agricultor:**
 - `POST /api/agricultor/post`
 - **Body:** nombres, apellidos, dpi, telefono, comunidadAldea, historiaPerfil, idLogin (FK).
- **Crear Perfil de Comprador:**
 - `POST /api/comprador/post`
 - **Body:** nombreComprador, apellidoComprador, telefonoComprador, direccionComprador, idLogin (FK).

5.2 Módulo de Inventario y Cosechas

- **Crear Categoría:**
 - `POST /api/categoria/post`
 - **Body:** nombreCategoria, descripcionCategoria
- **Publicar un Producto:**
 - `POST /api/producto/post`
 - **Body:** nombreProducto, descripcionProducto, precioProducto, stockProducto, fechaCosechaProducto, idAgricultor (FK), idCategoria (FK).
- **Consultar Productos:**
 - `GET /api/producto/get` (Retorna la lista completa de inventario con sus relaciones).

5.3 Módulo de Transacciones y Valoraciones

- **Generar un Pedido:**
 - `POST /api/pedido/post`
 - **Body:** totalPedido, cantidadPedido, estadoPedido, idComprador (FK), idProducto (FK).
- **Emitir Factura:**
 - `POST /api/factura/post`
 - **Body:** nitfactura, fechaEmisionfac, totalfactura, idPedido (FK).
- **Registrar Valoración (Review):**

- `POST /api/valoracion/post`
- **Body:** `puntuacionValoracion`, `comentarioValoracion`, `fechaValoracion`, `idProducto` (FK), `idComprador` (FK).

Nota: Además de los métodos POST, la API soporta métodos GET (listar/obtener por ID), PUT (actualizar registros) y DELETE (eliminar registros) para todas las entidades mediante sus respectivos controladores y repositorios en Spring Boot.

6. MANEJO DE ERRORES Y TROUBLESHOOTING (DEPURACIÓN)

Durante la fase de integración y pruebas en Postman, el equipo resolvió satisfactoriamente múltiples desafíos técnicos propios de arquitecturas robustas:

1. **Resolución de Constraint Violations (Error 1452):** Se depuraron y solucionaron errores de *Foreign Key Constraint Fails* sincronizando correctamente los IDs generados de forma autoincremental en MySQL con los Payloads JSON enviados desde el cliente, garantizando la integridad referencial de los pedidos.
 2. **Manejo de Respuestas de Serialización:** Se interceptaron advertencias del tipo `HttpMessageNotWritableException`, comprendiendo el comportamiento asíncrono de Hibernate al retornar objetos anidados, asegurando que los datos se persistieran correctamente en la base de datos a pesar de las limitaciones de visualización del JSON de retorno.
 3. **Refactorización de Entidades y Columnas (Error 1048 - Column cannot be null):** Se realizó una auditoría de código en las clases de Entidad (ej. `Comprador.java`, `Valoracion.java`), corrigiendo discrepancias de nomenclatura (Typing) entre las variables de Java (Getters/Setters) y las columnas en MySQL (ej. corrección de `fechaValoracion` a `fechaValoracion`), eliminando por completo los errores `Internal Server Error 500`.
-

7. CONCLUSIÓN

La fase de Backend de "El Surco" ha sido concluida exitosamente. El equipo demostró dominio en la creación de aplicaciones corporativas en Java, manejo de inyección de dependencias con Spring, persistencia de datos con JPA/Hibernate y control estricto de bases de datos relacionales. La API REST se encuentra estable, documentada y lista para ser consumida por la futura interfaz de usuario (Frontend).